

Engineering write-up

SUPPORT TICKET TRIAGE SERVICE | TAKE-HOME SUBMISSION

1. Architecture and why

My main goal was to keep this project simple and easy to run. I initially started with Next.js because I was considering a frontend as well. After reviewing the scope, I decided that an API and a terminal chat client were enough. I switched to Express and kept TypeScript. There are only a few endpoints, so Express makes the request flow easy to follow without carrying the frontend setup.

I chose SQLite because this project needs persistent data, but does not need a separate database server. A reviewer can start the application without setting up another service. Messages, decisions and tool attempts stay available after a restart. This version is intended to run as one local service; I have not designed it for multiple servers.

I wanted to try Jev because much of triage is choosing between known options: urgency, issue type and which FAQ is relevant. The idea was to let Jev handle those judgments and let GPT write the conversation reply. Reducing GPT usage and cost was also a motivation, but the small tests do not establish a cost advantage. Jev adds another provider to configure, so I kept it optional. Reviewers can use OpenAI alone with their own key, as the assignment requires.

There are only six mock FAQ articles. I send them with the conversation for assessment instead of adding embeddings and a vector database. Zod checks incoming data and model responses. Application code decides which actions are allowed.

2. What I focused on and left out

I focused on the complete flow: receive a ticket, read earlier messages, make a decision, use the allowed tools and save the result. Simple questions with enough FAQ support can receive an automatic answer. Billing and bug reports are recommended for specialist review; uncertain cases go to a human. These recommendations do not actually notify a team.

The two tools are FAQ search and opening a simulated incident. The incident tool writes to its own database. It is allowed only with strong evidence of an ongoing outage affecting several users. There are no tools for refunds or account changes.

Safe retries were a priority. Each request has a key tied to its contents. The service saves its decision before opening an incident, and the incident provider also remembers the operation key. If the incident is saved but its response is lost, retrying finds the original instead of creating another. This depends on the provider keeping that record too. Completed requests return their saved response.

I left out authentication, deployment, streaming, incident closing/reopening and merging related tickets. With another week, I would improve failed-request recovery, add more evaluation cases and work on the reply problems described below.

3. Problems in the sample tickets

Billing: three entries in a bank app do not prove three completed payments. The missing Pro access and presentation deadline make this high priority even for a Free customer. Billing needs to verify payments and account access. The assistant must not promise a refund or tell the customer to postpone bank disputes. Some generated wording still goes beyond the evidence; a few text checks cannot catch every version of that mistake.

Thai outage: several coworkers failing across browsers is a reason to investigate even if the status page is green. Having 45 seats does not prove that all 45 people are affected. The code requires enough evidence before creating an incident; otherwise it asks for human review. The reply should not invent a cause or repair time. Only the incident integration is simulated, not the customer's reported problem.

Theme: asking about scheduling does not mean the earlier System Default bug is fixed. The case should remain medium priority with an engineering recommendation. OS scheduling is useful only after theme synchronization works. Testing also exposed English questions receiving Thai replies. Reply language now follows the current human message, while extracted customer language and sentiment still come from customer messages. Earlier assistant answers are excluded from new model evidence.

4. Testing and evaluation

Tests cover the rules, input validation, repeated requests and failures around incident creation. An HTTP test saves an incident, loses the response, restarts the service and retries. Offline evaluation uses labelled cases with a mock; live evaluation runs the actual providers. The results and remaining failures are in `VALIDATION.md`.

Passing these checks does not mean every answer is correct. Some replies still add unsupported advice. Common wrong-language and false-action drafts are checked; cases already requiring a human can receive a marked standard handoff message instead. This does not solve every wording problem.

In production, I would measure missed urgent cases, unnecessary escalations, FAQ usefulness, incorrect claims, language consistency, response time and cost. I would also track how often operators change a decision, review examples by language and issue type, and test prompt/model changes on cases not used during development. Saved inputs, decisions and tool results help explain what happened when something goes wrong.